Vulnerability Management Procedures

App: Time on the record (Zoom App)
Owner / security contact: danielbermudez6one9@gmail.com
Last reviewed: 2026-10-17 · **Review cadence:** at least every 12 months

1. Purpose

Defines how vulnerabilities in the App's code, dependencies, and hosting are found, assessed, fixed, and verified.

2. Sources of vulnerability information

Source	Frequency
SAST: `npm run lint:security`	Every release, and before every commit that touches `server.js` or `public/*.js`
SCA: `npm audit`	Every release, and at least monthly
DAST: OWASP ZAP active scan	Every release that changes endpoints, headers, or pages; at least quarterly
GitHub Dependabot alerts on the source repository	Continuous; alerts are reviewed within 2 business days
Zoom developer security notices	As published
External reports	Any time, to danielbermudez6one9@gmail.com (also listed on the App's Support page)

3. Assessment

Each finding is logged, with tool, date, affected component, and severity, in the repository's issue tracker or in `security/README.md`. Severity is assigned by:

1. the CVSS v3/v4 base score from the advisory when one exists;
2. otherwise, the scanner's risk rating;
3. then adjusted for real exposure in this App. For example, a vulnerable code path that the App never calls may be lowered one level. **The reason for any adjustment is written down.**

4. Remediation timelines

Severity	CVSS	Fix deployed within
Critical	9.0–10.0	7 days (24 hours if actively exploited)
High	7.0–8.9	30 days
Medium	4.0–6.9	90 days
Low / Informational	< 4.0	Next scheduled release, or accepted with a written reason

The clock starts when the finding is confirmed.

5. Remediation

- **Dependencies:** upgrade to a fixed version (`npm audit fix` or a targeted `npm install <pkg>@<version>`). Re-run the full test and scan suite. Commit the updated `package-lock.json`.
- **Code:** fix the root cause, not only the reported instance. Add a lint rule or test where practical so the issue can't come back.
- **Configuration and headers:** change `server.js` middleware, then confirm with a DAST re-scan.

6. Verification and closure

A finding is closed only after the scan that found it (or an equivalent manual check) is re-run and no longer reports it. The re-run output is saved to `security/evidence/`.

7. Risk acceptance

If a finding cannot be fixed within its timeline (for example, no upstream fix exists), the owner records:

- the finding;
- why it can't be fixed yet;
- the compensating controls;
- a review date of no more than 90 days.

False positives are recorded the same way, with the reasoning.

8. Example: October 2026 remediation cycle

- `npm audit` reported high-severity advisories in `axios`, `brace-expansion`, and `proxy-addr` (`evidence/sca-npm-audit-before-remediation.txt`). These were fixed by dependency upgrades in commit `f1e1684`. The re-scan reports 0 vulnerabilities (`evidence/sca-npm-audit.txt`).
- An OWASP ZAP scan reported 3 medium and 1 low alert: missing CSP `frame-ancestors` (clickjacking), no Subresource Integrity on the Zoom SDK script, and cross-domain script inclusion. All were fixed by

adding `frame-ancestors`, keeping security headers on 404 responses, and serving the Zoom SDK from the pinned npm package. The re-scan reports 0 high/medium/low (`evidence/dast-zap-report.html`).
- A SAST "object injection" warning in the PKCE verifier generator was a false positive (indexing a constant string with a random byte). The code was rewritten so the warning no longer applies, rather than suppressed.
